

Project Proposal: Resilient Visual-Inertial Navigation Stack for UAS

1. Overview

Goal: Design, implement, and validate a real-time visual-inertial navigation (VIO/SLAM) system for small UAS that maintains accurate pose estimates when GNSS is degraded, intermittent, or unavailable, then deploy the full pipeline on embedded flight-relevant hardware.

Scope: A trimmed, demo-ready version of the full navigation stack effort — estimator core, GNSS fusion with graceful dropout handling, lean ROS2 integration, and embedded deployment. The learned (deep-learning) perception front end and the full alternative-PNT stretch goal are scoped out to fit a 3-month part-time timeline, and are noted as natural extensions rather than core deliverables.

Estimated duration: ~12–14 weeks part-time (~8–12 hrs/week), i.e. roughly 3 months.

Why this project: It's a self-contained, demoable slice of the full stack that still exercises perception, estimation, sensor fusion under degraded conditions, real-time software engineering, and embedded deployment — the core of what the posting asks for — without requiring the deep-learning and full alt-PNT scope that would push it past a 3-month window.

2. Technical Approach

Layer	Approach
Perception front end	Classical feature tracking (ORB/FAST) — sufficient for a credible, complete demo
State estimation	Error-state EKF fusing camera + IMU (VIO), extended to loosely-coupled GNSS correction
Fallback logic	Automatic degradation to VIO-only estimate when GNSS quality drops below threshold
Middleware	ROS2 node wrapping the estimator, publishing standard <code>nav_msgs/Odometry</code> , visualized in RViz

Layer	Approach
Deployment target	NVIDIA Jetson Orin Nano (or similar edge GPU board)
Languages	Python for prototyping/analysis, C++ for the real-time estimator

Explicitly out of scope (noted as future extensions): a learned/deep-learning perception front end (e.g. SuperPoint) as a drop-in replacement for the classical detector, a second alternative-PNT correction source (barometer/magnetometer) beyond GNSS, and full Nav2 planner integration. These are natural "Phase 6+" additions if time allows, and are worth mentioning in a writeup or interview as designed-for-but-not-yet-built.

3. Phase-by-Phase Plan

Phase 1 — Estimator Core (4–5 weeks part-time)

Objective: Build and validate a working visual-inertial odometry estimator against ground truth.

1. Set up development environment (Python + OpenCV + a filtering library, e.g. `filterpy`, or hand-roll the EKF).
2. Download and inspect a public VIO benchmark dataset (EuRoC MAV or TUM VI) — understand its camera/IMU calibration format and ground-truth trajectory format.
3. Implement IMU pre-integration (propagate orientation, velocity, position between camera frames).
4. Implement visual front end: feature detection (FAST/ORB), tracking (KLT or descriptor matching), and outlier rejection (RANSAC).
5. Implement the EKF update step fusing visual feature observations with the IMU-propagated state.
6. Run the pipeline against the benchmark dataset and compute standard error metrics (Absolute Trajectory Error, Relative Pose Error).
7. Port the estimator's core loop from Python to C++ for performance, keeping Python for analysis/plotting.
8. Document baseline accuracy numbers — this becomes your comparison point for every later phase.

Deliverable: A working C++ VIO estimator with quantified accuracy on a public dataset.

Phase 2 — GNSS Fusion & Dropout Handling (3 weeks part-time)

Objective: Add GNSS as a correction source and handle its loss gracefully. This is the core scope — the single strongest piece of evidence for sensor fusion under degraded PNT.

1. Add a GNSS measurement model to the EKF (loosely-coupled: GNSS position/velocity as an update, not a hard reset).
2. Source or simulate a GNSS-augmented dataset (synthesize GNSS from ground truth plus injected noise/dropout if your dataset lacks it natively).
3. Implement a GNSS quality monitor (e.g. simulated HDOP/satellite-count threshold) that flags degraded fixes.
4. Implement fallback logic: when GNSS quality drops below threshold, the estimator reverts to VIO-only propagation without a hard fault.
5. Design and run a dropout test: inject 10–60 second GNSS blackouts at various points in a trajectory and measure drift accumulated during blackout.
6. Document drift-vs-blackout-duration results — this is your strongest, most demoable piece of evidence, and the centerpiece of your final writeup.

Deliverable: An estimator that demonstrably degrades gracefully under GNSS loss, with quantified drift curves.

Phase 3 — ROS2 Integration (2 weeks part-time)

Objective: Turn the standalone estimator into a reusable, visualizable navigation node. (Kept lean — no full planner integration.)

1. Set up a ROS2 workspace and package structure for the estimator.
2. Wrap sensor inputs (camera, IMU, GNSS) as ROS2 subscribers using standard message types (`sensor_msgs/Image`, `sensor_msgs/Imu`, `sensor_msgs/NavSatFix`).
3. Wrap the estimator as a node publishing `nav_msgs/Odometry` at a fixed rate.
4. Add TF2 broadcasting so the estimate is usable by other ROS2 tooling.
5. Validate the full ROS2 graph end-to-end in RViz using recorded bag data, including a live GNSS-dropout demo — this is the version of Phase 2's results that's actually visual and demoable in real time.

Deliverable: A ROS2 package publishing real-time odometry, with a live RViz demo of the GNSS-dropout fallback behavior.

Phase 4 — Embedded Deployment (3–4 weeks part-time)

Objective: Get the full pipeline running in real time on flight-relevant embedded hardware. This is the most differentiating deliverable — keep it, even under time pressure.

1. Set up the Jetson (or chosen board) with JetPack, ROS2, and required libraries; build the C++ estimator natively on-device.
2. Profile the unoptimized pipeline on-device: measure per-stage latency (feature extraction, EKF update, total loop rate).
3. Identify bottlenecks in the perception or estimation stage.
4. Optimize: multi-thread perception and estimation stages, reduce unnecessary copies/allocations in the hot path, tune parameters for the embedded CPU/GPU budget.
5. Re-profile and document before/after latency and achieved frame rate against a defined real-time budget (e.g. 30 Hz).
6. Run the full ROS2 pipeline on-device against recorded bag data (camera + IMU + GNSS playback, including a dropout segment) to confirm functional correctness matches desktop results.
7. Stress-test: run continuously for an extended period to check for memory leaks or thermal throttling on the embedded board.

Deliverable: The complete navigation stack running in real time on embedded hardware, with documented performance numbers.

4. Tools & Resources Needed

- Public datasets: EuRoC MAV, TUM VI (free, no hardware required to start)
 - Python: NumPy, OpenCV, `filterpy` or custom EKF
 - C++: Eigen (linear algebra), OpenCV C++ bindings
 - ROS2 (Humble or newer)
 - Embedded target: Jetson Orin Nano or comparable edge GPU board
-

5. Success Metrics

- Phase 1: ATE/RPE on benchmark dataset within a documented, competitive range of published VIO baselines.
- Phase 2: Quantified position drift (meters) as a function of GNSS blackout duration.
- Phase 3: Working ROS2 odometry stream visualized live in RViz, including a dropout-fallback demo.
- Phase 4: Real-time performance (defined Hz target) sustained on embedded hardware, with before/after optimization numbers.

6. Timeline Summary

Phase	Duration (part-time, ~8–12 hr/wk)
1. Estimator core	4–5 wk
2. GNSS fusion & dropout handling	3 wk
3. ROS2 integration	2 wk
4. Embedded deployment	3–4 wk
Total	~12–14 weeks (~3 months)

7. Demo Plan

For the final demo, show: (1) the estimator's trajectory overlaid against ground truth on the benchmark dataset, (2) a live GNSS-dropout test in RViz where the position estimate visibly holds steady on VIO alone during blackout and re-converges when GNSS returns, and (3) the same pipeline running on the Jetson in real time with on-screen latency/frame-rate numbers proving it meets the real-time budget. This sequence tells a complete story — accuracy, resilience, and deployability — in about five minutes.

8. Future Extensions (out of scope, worth mentioning)

- Swap the classical feature front end for a learned detector (e.g. SuperPoint) and compare accuracy/latency tradeoffs.
- Add a second alternative-PNT correction source (barometric altitude or magnetometer heading).
- Integrate a planner (Nav2) so the odometry estimate actually drives navigation, not just localization.
- Validate against a real flight log if UAS hardware access becomes available.